



Manual Administrativo SimpliaLeads

Informacion documentada controlada para operacion, mantenimiento y replicacion

1. Control documental

Campo	Valor
Codigo documental	MAN-ADM-SIMPLIA-001
Nombre	Manual Administrativo SimpliaLeads
Version	1.1
Fecha de emision	04/05/2026
Estado	Borrador tecnico / Vigente al aprobarse
Dueno del documento	Simplia - Area de tecnologia / administracion del producto
Aprobador	Direccion Simplia o responsable autorizado del proyecto
Proxima revision	04/11/2026
Zona horaria	America/Guayaquil
Distribucion	Administradores internos, desarrolladores, soporte tecnico y responsables de implementacion

Version	Fecha	Cambio	Responsable
1.0	02/05/2026	Emission inicial del manual administrativo.	Simplia
1.1	04/05/2026	Actualizacion de Calidad a 3 niveles, guia completa de replicacion, Vercel, Supabase, Chatwoot, Resend y anexos SQL ampliados.	Simplia

2. Objetivo, alcance y uso del manual

Este manual define como administrar, mantener, desplegar y replicar SimpliaLeads. Esta orientado a personal interno de Simplia, administradores de cuenta, soporte tecnico y desarrolladores que deban dejar funcionando el dashboard para una empresa.

Incluye	No incluye
Frontend React/Vite, Supabase schema cw, Edge Functions, Chatwoot, Resend, Vercel, reportes, cron, configuracion admin, validaciones y troubleshooting.	Capacitacion comercial para usuario final, secretos reales, credenciales privadas o decisiones comerciales especificas de una empresa.

- Usar este documento antes de crear un nuevo entorno, despues de un despliegue y durante soporte.
- Mantenerlo actualizado cuando cambien etiquetas, funciones, variables, reportes o estructura de Supabase.
- No pegar credenciales reales dentro del manual. Usar siempre placeholders entre <...>.

3. Contexto del sistema

Componente	Uso	Control administrativo
Frontend React/Vite	Dashboard visual: Estrategia, Embudo, Operacion, Seguimiento, Rendimiento Humano, Tendencias, Calidad, Conversaciones y Reportes.	Vercel, variables VITE_ y rama main.
Supabase	Base historica, estado actual, tablas cw, Edge Functions, Vault y cron.	SQL Editor, funciones, secretos, logs y migraciones.
Chatwoot	Fuente de conversaciones, contactos, mensajes, etiquetas, atributos y canales.	Cuenta, token API, webhooks, atributos personalizados y etiquetas.

Componente	Uso	Control administrativo
Resend	Envio de reportes programados por correo.	Dominio/remitente verificado, API key y FROM email.
Reportes	Exportaciones manuales y programadas en Excel, PDF y CSV.	Configuracion en pestana Reportes y tabla cw.automated_reports.

3.1 Flujo de datos

- Chatwoot conserva la operacion diaria: conversaciones, mensajes, etiquetas y atributos.
- El dashboard combina datos en vivo de periodos recientes con historico guardado en Supabase.
- Supabase guarda estado actual en tablas *_current y evidencia historica en tablas *_history, raw_ingest, sync_runs y report runs.
- El cron nocturno ejecuta la sincronizacion diaria a las 12:01 a. m. Ecuador mediante schedule UTC 1 5 * * *.
- Los reportes programados se revisan cada 5 minutos con el cron send-scheduled-reports.

3.2 Reglas de estado actual e historico

Los KPIs del dashboard deben representar lo que existe actualmente en Chatwoot/Supabase. Los cambios historicos sirven para explicar variaciones relevantes, pero no deben sumar ventas, citas o montos si ya no cumplen la condicion actual.

4. Roles y responsabilidades

Rol	Responsabilidad	Evidencia esperada
Admin Simplicia	Configura etiquetas, rangos, columnas, perfiles y reportes.	Cambios en panel admin y dashboard_tag_settings.
Desarrollador	Mantiene codigo, migraciones, Edge Functions y despliegues.	Commits, deploys, logs y manual actualizado.
Admin Supabase	Gestiona proyecto, secretos, cron, RLS, funciones y backups.	SQL aplicado, funciones activas, sync_runs.
Admin Chatwoot	Crea cuenta, inboxes, atributos, etiquetas, API token y webhooks.	Configuracion visible en Chatwoot.
Responsable Resend	Verifica dominio/remitente y crea API key.	Email de prueba y corridas success.
Usuario operativo	Usa dashboard y reportes sin modificar backend.	Exportaciones y acciones comerciales.

5. Configuracion admin del dashboard

El panel admin permite adaptar el dashboard a la forma de trabajo de cada empresa sin cambiar codigo. La configuracion se guarda en cw.dashboard_tag_settings dentro del objeto settings.

Configuracion	Que define	Impacto
Embudo	Estados/etiquetas comerciales y orden de lectura.	Embudo, estrategia, seguimiento y reportes.
Citas	Estados que cuentan como cita o avance comercial.	Conversion a cita, rendimiento humano y seguimiento.
Ventas	Etiquetas que cuentan como venta actual.	Ventas, monto, ticket promedio y reportes gerenciales.
Descalificacion	Estados que agrupan leads que no aplican.	Tendencias y motivos de perdida.
Calidad	Campo numerico y rangos Caliente/Tibio/Frio.	KPIs, tabla de leads evaluados, filtros y reportes.
Reportes	Columnas, formatos, destinatarios, perfiles y frecuencia.	Exportaciones manuales y envios automaticos.

5.1 Calidad de leads - regla vigente

La clasificacion vigente usa tres niveles. Frio es el nivel base; por eso todo lead sin puntaje o con puntaje menor a Tibio se clasifica como Frio.

Nivel	Rango	Interpretacion
Caliente	Puntaje mayor o igual a 70	Prioridad comercial alta. Requiere seguimiento inmediato.

Nivel	Rango	Interpretacion
Tibio	Puntaje mayor o igual a 45 y menor a 70	Tiene senales de interes. Requiere nutrir y llevar a cita.
Frio	Puntaje menor a 45 o sin puntaje	Base de clasificacion. Requiere revisar contexto antes de priorizar.

- Inputs visibles en admin: Desde Caliente y Desde Tibio.
- Defaults recomendados: Caliente desde 70, Tibio desde 45.
- scoreThresholds debe guardar hotMin y warmMin. Si existe coldMin antiguo, se ignora para clasificacion nueva.
- Exportes manuales y programados deben resumir solo Caliente, Tibio y Frio.

6. Logica funcional por pestana

Pestana	Datos principales	Puntos de control
Estrategia	Total leads, calificados, citas, ventas, conversion y ganancias.	Debe coincidir con filtros de fecha/canal y reglas admin.
Embudo	Etapas actuales, historico, conversiones y perdidas.	Usa etiquetas configuradas y estado actual.
Operacion	Tiempo de respuesta, leads sin respuesta, responsables y carga.	Lead sin respuesta: ultima interaccion real del cliente sin respuesta posterior.
Seguimiento	Pendientes, citas, ventas y acciones proximas.	Excel de esta pestana conserva formato especifico existente.
Rendimiento Humano	Responsables, citas humanas, ventas, conversion, total vendido y ticket promedio.	Ventas actuales requieren etiqueta vigente y monto mayor a 0.
Tendencias	Canales, campanas, ingresos, descalificacion y comportamiento por periodo.	Descalificacion se presenta en lenguaje simple.
Calidad	Caliente, Tibio, Frio, puntaje, campana y leads evaluados.	Sin puntaje se clasifica como Frio.
Conversaciones	Tabla de conversaciones, historial, estado, canal, URL y ultima interaccion.	Excel conserva formato especifico existente.
Reportes	Descargas manuales y reportes programados.	Usa modelo compartido manual/programado.

7. Replicar SimpliaLeads para otra empresa

Esta seccion es el procedimiento maestro para levantar un entorno nuevo. Debe ejecutarse en orden y registrar evidencias de cada paso.

Paso	Actividad	Resultado esperado
1	Crear o confirmar repositorio GitHub del proyecto y rama main.	Codigo fuente disponible para Vercel.
2	Crear proyecto Supabase nuevo.	URL, anon key y service role key disponibles.
3	Ejecutar SQL base y migraciones.	Schema cw creado con tablas, vistas, indices, RLS y grants.
4	Desplegar Edge Functions.	Funciones activas en Supabase.
5	Crear/configurar cuenta Chatwoot.	Account ID, API token, inboxes, etiquetas, atributos y webhook listos.
6	Configurar Resend.	Dominio/remitente verificado y API key guardada.
7	Configurar Vercel desde GitHub.	Dashboard desplegado desde main.
8	Agregar variables en Vercel y secretos en Supabase.	Frontend y Edge Functions con credenciales correctas.
9	Ejecutar sync inicial y reparacion si aplica.	Datos visibles en dashboard.
10	Configurar panel admin y reportes.	Reglas comerciales adaptadas a la empresa.

7.1 Que se cambia entre empresas

- Supabase: proyecto, URL, anon key, service role key, secretos y cron.
- Chatwoot: base URL si no es Chatwoot Cloud, account ID, token API, inboxes, atributos, etiquetas y webhook secret.
- Resend: API key, dominio, remitente y destinatarios de reportes.
- Vercel: variables de entorno y proyecto conectado al repo/branch correcto.
- Admin dashboard: estados de embudo, citas, ventas, descalificacion, calidad, columnas y reportes programados.

8. Despliegue en Vercel

El frontend se despliega en Vercel desde GitHub. Para la operacion actual de Simplia, ingresar a Vercel con Google/Gmail usando simplia.social@gmail.com y seleccionar el proyecto conectado al repositorio.

Campo	Configuracion
Cuenta de acceso	Login con Google/Gmail: simplia.social@gmail.com
Origen	GitHub
Rama productiva	main
Framework	Vite
Build command	npm run build
Output directory	dist
Archivo de rutas	vercel.json

8.1 Variables de entorno de Vercel

Configurar estas variables en Vercel > Project Settings > Environment Variables. No usar valores reales dentro de este manual.

Ejemplo .env para Vercel

```
# Variables de entorno en Vercel - Project Settings > Environment Variables
VITE_SUPABASE_URL=https://<PROJECT_REF>.supabase.co
VITE_SUPABASE_ANON_KEY=<SUPABASE_ANON_KEY>
VITE_CHATWOOT_BASE_URL=https://app.chatwoot.com
VITE_CHATWOOT_ACCOUNT_ID=<CHATWOOT_ACCOUNT_ID>
VITE_CHATWOOT_API_TOKEN=<CHATWOOT_API_TOKEN>
```

8.2 Proxy Chatwoot

El archivo vercel.json reescribe /chatwoot-api hacia Chatwoot. Si una empresa usa Chatwoot Cloud se mantiene https://app.chatwoot.com. Si usa Chatwoot self-hosted, cambiar el destination a la URL publica de esa instancia.

vercel.json esperado

```
{
  "rewrites": [
    {
      "source": "/chatwoot-api/:path*",
      "destination": "https://app.chatwoot.com/:path*"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

8.3 Validacion de deploy

- Abrir la URL de Vercel y confirmar que carga el logo y las pestanas.
- Confirmar que no hay errores de variables faltantes en consola.
- Revisar que los filtros de fecha/canal funcionen.
- Probar una exportacion manual despues de cargar datos.

9. Supabase paso a paso

Paso	Accion	Explicacion
1	Crear proyecto Supabase.	Genera URL, anon key, service role key y base Postgres.
2	Habilitar extensiones.	pgcrypto crea UUIDs, pg_net llama Edge Functions, pg_cron agenda tareas y Vault guarda secretos.
3	Crear schema cw.	Aisla toda la data de Chatwoot/dashboard.
4	Crear tablas base.	Estado actual, historicos, mensajes, configuracion y reportes.
5	Aplicar migraciones.	Actualiza vistas, indices, cron, cambios relevantes y reportes programados.
6	Configurar RLS/grants.	Permite lectura/escritura controlada al frontend autenticado y service_role.
7	Guardar secretos.	Permite que Edge Functions y cron funcionen sin exponer claves.
8	Crear cron.	Ejecuta sync diario y reportes programados.
9	Deploy Edge Functions.	Activa integracion con Chatwoot y Resend.
10	Ejecutar sync inicial.	Carga datos para que dashboard deje de aparecer vacio.

9.1 Script base de creacion

Este bloque crea la estructura base necesaria del schema cw. En un proyecto nuevo, ejecutar primero este bloque y luego las migraciones/versiones de actualizacion.

SQL base desde cero

```
-- =====
-- BASE DESDE CERO - SCHEMA CW PARA SIMPLIALEADS
-- Ejecutar en Supabase SQL Editor antes de desplegar funciones.
-- Sustituir valores entre <...> por los datos del nuevo proyecto.
-- =====

create extension if not exists pgcrypto with schema extensions;
create extension if not exists pg_net with schema extensions;
create extension if not exists supabase_vault with schema vault;
create extension if not exists pg_cron with schema pg_catalog;

create schema if not exists cw;

create table if not exists cw.account_config (
  id uuid primary key default gen_random_uuid(),
  company_name text not null,
  chatwoot_base_url text not null,
  chatwoot_account_id bigint not null unique,
  timezone text not null default 'America/Guayaquil',
  notes text,
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);

create table if not exists cw.sync_cursor (
  cursor_name text primary key,
  last_since_ts timestampz,
  last_until_ts timestampz,
  cursor_payload jsonb not null default '{} '::jsonb,
  updated_at timestampz not null default now()
);

create table if not exists cw.sync_runs (
  id bigserial primary key,
  sync_type text not null,
  started_at timestampz not null default now(),
  finished_at timestampz,
  status text not null default 'running'
  check (status in ('running', 'success', 'partial', 'error')),
  stats jsonb not null default '{} '::jsonb,
  error_message text,
  created_at timestampz not null default now()
);

create table if not exists cw.raw_ingest (
  id bigserial primary key,
  source_type text not null check (source_type in ('api', 'webhook', 'manual', 'repair')),
  endpoint_name text not null,
  event_name text,
  entity_type text,
  chatwoot_entity_id bigint,
  payload jsonb not null,
  fetched_at timestampz not null default now(),
  processed boolean not null default false,
  processing_error text
);

create table if not exists cw.inboxes (
  chatwoot_inbox_id bigint primary key,
  name text,
  website_url text,
```

```

        channel_type text,
        avatar_url text,
        widget_color text,
        website_token text,
        enable_auto_assignment boolean,
        web_widget_script text,
        welcome_title text,
        welcome_tagline text,
        greeting_enabled boolean,
        raw_payload jsonb not null default '{} '::jsonb,
        created_at timestampz not null default now(),
        updated_at timestampz not null default now()
    );

create table if not exists cw.teams (
    chatwoot_team_id bigint primary key,
    name text,
    description text,
    allow_auto_assign boolean,
    is_member boolean,
    raw_payload jsonb not null default '{} '::jsonb,
    created_at timestampz not null default now(),
    updated_at timestampz not null default now()
);

create table if not exists cw.attribute_definitions (
    id bigserial primary key,
    chatwoot_attribute_id bigint not null unique,
    attribute_scope text not null check (attribute_scope in ('contact', 'conversation', 'unknown')),
    attribute_key text not null,
    attribute_display_name text,
    attribute_display_type text,
    attribute_description text,
    regex_pattern text,
    regex_cue text,
    attribute_values jsonb,
    attribute_model text,
    default_value jsonb,
    created_at_chatwoot timestampz,
    updated_at_chatwoot timestampz,
    raw_payload jsonb not null default '{} '::jsonb,
    created_at timestampz not null default now(),
    updated_at timestampz not null default now()
);

create table if not exists cw.contacts_current (
    chatwoot_contact_id bigint primary key,

    lead_identity_key text,
    identifier text,
    name text,
    phone_number text,
    email text,
    blocked boolean,
    thumbnail text,
    availability_status text,
    additional_attributes jsonb not null default '{} '::jsonb,
    custom_attributes jsonb not null default '{} '::jsonb,
    created_at_chatwoot timestampz,
    last_activity_at_chatwoot timestampz,
    first_seen_at timestampz not null default now(),
    last_seen_at timestampz not null default now(),
    raw_payload jsonb not null default '{} '::jsonb,
    created_at timestampz not null default now(),
    updated_at timestampz not null default now()
);

create table if not exists cw.contact_inboxes (
    id bigserial primary key,
    chatwoot_contact_id bigint not null references cw.contacts_current(chatwoot_contact_id) on delete
cascade,
    chatwoot_inbox_id bigint,
    source_id text,
    inbox_name text,
    channel_type text,
    provider text,
    raw_payload jsonb not null default '{} '::jsonb,
    created_at timestampz not null default now(),
    updated_at timestampz not null default now()
);

create table if not exists cw.conversations_current (
    chatwoot_conversation_id bigint primary key,
    chatwoot_contact_id bigint references cw.contacts_current(chatwoot_contact_id) on delete set null,
    chatwoot_account_id bigint,
    chatwoot_inbox_id bigint,
    chatwoot_team_id bigint,
    assignee_id bigint,
    uuid text,
    status text,
    priority text,
    can_reply boolean,
    muted boolean,
    snoozed_until timestampz,
    unread_count integer,
    labels text[] not null default '{} '::text[],
    business_stage_current text,
    additional_attributes jsonb not null default '{} '::jsonb,
    custom_attributes jsonb not null default '{} '::jsonb,
    conversation_custom_attributes jsonb not null default '{} '::jsonb,

```

```

contact_custom_attributes jsonb not null default '{}':jsonb,
meta jsonb not null default '{}':jsonb,

applied_sla jsonb not null default '{}':jsonb,
sla_events jsonb not null default '[]':jsonb,
inbox_name text,
channel_type text,
provider text,
conversation_url text,
timestamp_text text,
first_reply_created_at_chatwoot timestampz,
agent_last_seen_at_chatwoot timestampz,
assignee_last_seen_at_chatwoot timestampz,
contact_last_seen_at_chatwoot timestampz,
waiting_since_chatwoot timestampz,
last_activity_at_chatwoot timestampz,
created_at_chatwoot timestampz,
updated_at_chatwoot timestampz,
last_non_activity_message_id bigint,
last_non_activity_message_preview text,
first_message_at timestampz,
last_message_at timestampz,
last_incoming_message_at timestampz,
last_outgoing_message_at timestampz,
total_messages integer not null default 0,
nombre_completo text,
fecha_visita text,
hora_visita text,
agencia text,
celular text,
correo text,
campana text,
ciudad text,
edad text,
canal text,
agente boolean,
score_interes numeric,
monto_operacion text,
fecha_monto_operacion timestampz,
perfil_url text,
raw_payload jsonb not null default '{}':jsonb,
created_at timestampz not null default now(),
updated_at timestampz not null default now()
);

create table if not exists cw.contact_attribute_history (
    id bigserial primary key,
    chatwoot_contact_id bigint not null references cw.contacts_current(chatwoot_contact_id) on delete
    cascade,
    attribute_key text not null,
    old_value jsonb,
    new_value jsonb,
    changed_at timestampz not null default now(),
    change_source text not null default 'sync' check (change_source in ('sync', 'webhook', 'repair',
'manual')),
    created_at timestampz not null default now(),
    event_key text
);

create table if not exists cw.conversation_attribute_history (
    id bigserial primary key,
    chatwoot_conversation_id bigint not null references
    cw.conversations_current(chatwoot_conversation_id) on delete cascade,
    attribute_key text not null,
    old_value jsonb,
    new_value jsonb,
    changed_at timestampz not null default now(),
    change_source text not null default 'sync' check (change_source in ('sync', 'webhook', 'repair',
'manual')),
    created_at timestampz not null default now(),
    event_key text
);

create table if not exists cw.conversation_label_history (
    id bigserial primary key,
    chatwoot_conversation_id bigint not null references
    cw.conversations_current(chatwoot_conversation_id) on delete cascade,
    label text not null,
    action text not null check (action in ('added', 'removed')),
    changed_at timestampz not null default now(),
    change_source text not null default 'sync' check (change_source in ('sync', 'webhook', 'repair',
'manual')),
    created_at timestampz not null default now()
);

create table if not exists cw.business_stage_history (
    id bigserial primary key,
    chatwoot_conversation_id bigint not null references
    cw.conversations_current(chatwoot_conversation_id) on delete cascade,
    old_stage text,
    new_stage text,
    changed_at timestampz not null default now(),
    change_reason text,
    change_source text not null default 'sync'
    check (change_source in ('sync', 'webhook', 'repair', 'manual', 'bot')),
    created_at timestampz not null default now()
);

create table if not exists cw.messages (

```

```

chatwoot_message_id bigint primary key,
chatwoot_conversation_id bigint not null references
cw.conversations_current(chatwoot_conversation_id) on delete cascade,
chatwoot_contact_id bigint,
chatwoot_account_id bigint,
chatwoot_inbox_id bigint,
sender_id bigint,
sender_type text,
message_type text,
message_direction text not null default 'unknown'
    check (message_direction in ('incoming', 'outgoing', 'activity', 'note', 'unknown')),
content text,

content_type text,
content_attributes jsonb not null default '{}':jsonb,
additional_attributes jsonb not null default '{}':jsonb,
external_source_ids jsonb not null default '{}':jsonb,
attachments jsonb not null default '[]':jsonb,
sender jsonb not null default '{}':jsonb,
sentiment jsonb not null default '{}':jsonb,
processed_message_content text,
source_id text,
status text,
is_private boolean not null default false,
created_at_chatwoot timestampz,
updated_at_chatwoot timestampz,
raw_payload jsonb not null default '{}':jsonb,
created_at timestampz not null default now()
);

create table if not exists cw.reporting_events (
    chatwoot_reporting_event_id bigint primary key,
    name text,
    value numeric,
    value_in_business_hours numeric,
    event_start_time timestampz,
    event_end_time timestampz,
    chatwoot_account_id bigint,
    chatwoot_conversation_id bigint,
    chatwoot_inbox_id bigint,
    chatwoot_user_id bigint,
    created_at_chatwoot timestampz,
    updated_at_chatwoot timestampz,
    raw_payload jsonb not null default '{}':jsonb,
    created_at timestampz not null default now()
);

create table if not exists cw.daily_metrics (
    id bigserial primary key,
    metric_date date not null,
    metric_scope text not null
        check (metric_scope in ('summary', 'activities', 'unique', 'stage', 'label', 'channel', 'inbox',
'team', 'sla', 'custom')),
    metric_name text not null,
    metric_value numeric not null,
    dimensions jsonb not null default '{}':jsonb,
    calculated_at timestampz not null default now(),
    dim_hash text generated always as (md5((dimensions)::text)) stored
);

create table if not exists cw.automated_reports (
    id uuid primary key default gen_random_uuid(),
    name text not null,
    frequency text not null default 'weekly',
    schedule_days text[] not null default '{}':text[],
    schedule_month_day integer,
    schedule_time time not null default '08:00:00',

    recipients text not null,
    is_active boolean default true,
    last_run_at timestampz,
    report_scope text not null default 'tab' check (report_scope in ('tab', 'critical_profile')),
    tab_ids text[] not null default '{}':text[],
    critical_profile_key text,
    file_formats text[] not null default array['excel']::text[],
    date_range_mode text not null default 'closed_period' check (date_range_mode in ('closed_period',
'current_filters')),
    filters jsonb not null default '{}':jsonb,
    created_by uuid references auth.users(id) on delete set null,
    created_by_email text,
    last_status text,
    last_error text,
    created_at timestampz default now(),
    updated_at timestampz default now()
);

create table if not exists cw.dashboard_tag_settings (
    id uuid primary key default gen_random_uuid(),
    account_id bigint unique,
    settings jsonb not null default '{}':jsonb,
    updated_at timestampz default now()
);

create table if not exists cw.conversation_label_events (
    id bigint generated by default as identity primary key,
    chatwoot_conversation_id bigint not null,
    previous_labels text[] not null default '{}':text[],
    next_labels text[] not null default '{}':text[],
    added_labels text[] not null default '{}':text[],
    removed_labels text[] not null default '{}':text[],

```

```

    event_source text not null default 'sync_diff'
    check (event_source in ('dashboard', 'webhook', 'sync_diff', 'repair')),
    occurred_at timestampz not null default now(),
    detected_at timestampz not null default now(),
    raw_payload jsonb not null default '{}::jsonb',
    event_key text unique,
    created_at timestampz not null default now()
);

create table if not exists cw.automated_report_runs (
    id uuid primary key default gen_random_uuid(),
    automated_report_id uuid references cw.automated_reports(id) on delete set null,
    status text not null default 'running' check (status in ('running', 'success', 'error', 'skipped')),
    recipients text,
    file_formats text[] not null default '{}::text[]',
    report_scope text,
    tab_ids text[] not null default '{}::text[]',
    critical_profile_key text,
    scheduled_for timestampz,
    started_at timestampz not null default now(),
    finished_at timestampz,
    error_message text,

    metadata jsonb not null default '{}::jsonb'
);

create index if not exists idx_cw_conversations_created_at on cw.conversations_current
(created_at_chatwoot desc);
create index if not exists idx_cw_conversations_last_activity on cw.conversations_current
(last_activity_at_chatwoot desc);
create index if not exists idx_cw_conversations_labels_gin on cw.conversations_current using gin
(labels);
create index if not exists idx_cw_conversations_attrs_gin on cw.conversations_current using gin
(custom_attributes);
create index if not exists idx_cw_messages_created_at on cw.messages (created_at_chatwoot desc);
create index if not exists idx_cw_messages_conversation on cw.messages (chatwoot_conversation_id,
created_at_chatwoot);
create unique index if not exists idx_cw_daily_metrics_unique on cw.daily_metrics (metric_date,
metric_scope, metric_name, dim_hash);
create index if not exists automated_reports_active_schedule_idx on cw.automated_reports (is_active,
frequency, schedule_time);
create index if not exists automated_report_runs_report_started_idx on cw.automated_report_runs
(automated_report_id, started_at desc);

alter table cw.account_config enable row level security;
alter table cw.sync_cursor enable row level security;
alter table cw.sync_runs enable row level security;
alter table cw.raw_ingest enable row level security;
alter table cw.inboxes enable row level security;
alter table cw.teams enable row level security;
alter table cw.attribute_definitions enable row level security;
alter table cw.contacts_current enable row level security;
alter table cw.contact_inboxes enable row level security;
alter table cw.conversations_current enable row level security;
alter table cw.contact_attribute_history enable row level security;
alter table cw.conversation_attribute_history enable row level security;
alter table cw.conversation_label_history enable row level security;
alter table cw.business_stage_history enable row level security;
alter table cw.messages enable row level security;
alter table cw.reporting_events enable row level security;
alter table cw.daily_metrics enable row level security;
alter table cw.automated_reports enable row level security;
alter table cw.dashboard_tag_settings enable row level security;
alter table cw.conversation_label_events enable row level security;
alter table cw.automated_report_runs enable row level security;

grant usage on schema cw to authenticated, service_role;
grant select, insert, update, delete on all tables in schema cw to authenticated;
grant all on all tables in schema cw to service_role;
grant usage, select on all sequences in schema cw to authenticated, service_role;

-- Politicas base. Para datos sensibles, endurecer por tenant antes de multiempresa real.
do $$
declare
    t text;
begin
    foreach t in array array[
        'account_config','sync_cursor','sync_runs','inboxes','teams','attribute_definitions',
        'contacts_current','contact_inboxes','conversations_current','messages','reporting_events',
        'daily_metrics','automated_reports','dashboard_tag_settings','conversation_label_events'
    ] loop
        if not exists (
            select 1 from pg_policies
            where schemaname = 'cw' and tablename = t and policyname = 'authenticated_all'
        ) then
            execute format(
                'create policy authenticated_all on cw.%I for all to authenticated using (true) with
check (true)',
                t
            );
        end if;
    end loop;
end $$;

```

9.2 Secretos de Supabase

Guardar secretos con Supabase CLI. Los valores entre <...> deben reemplazarse por los valores reales del nuevo proyecto.

Comandos de secretos

```
npx supabase secrets set SUPABASE_URL=https://<PROJECT_REF>.supabase.co
npx supabase secrets set SUPABASE_SERVICE_ROLE_KEY=<SUPABASE_SERVICE_ROLE_KEY>
npx supabase secrets set VITE_CHATWOOT_BASE_URL=https://app.chatwoot.com
npx supabase secrets set VITE_CHATWOOT_ACCOUNT_ID=<CHATWOOT_ACCOUNT_ID>
npx supabase secrets set VITE_CHATWOOT_API_TOKEN=<CHATWOOT_API_TOKEN>
npx supabase secrets set CHATWOOT_WEBHOOK_SECRET=<WEBHOOK_SECRET_GENERADO>
npx supabase secrets set RESEND_API_KEY=<RESEND_API_KEY>
npx supabase secrets set RESEND_FROM_EMAIL="Simplia Reportes <reportes@empresa.com>"
```

9.3 Deploy de Edge Functions

Comandos de deploy

```
npx supabase login
npx supabase link --project-ref <PROJECT_REF>
npx supabase functions deploy chatwoot-sync
npx supabase functions deploy chatwoot-label-webhook
npx supabase functions deploy chatwoot-repair-conversations
npx supabase functions deploy send-scheduled-reports
```

9.4 Sync inicial y pruebas

Comandos de prueba

```
npx supabase functions invoke chatwoot-sync --body
'{"mode":"full","window_hours":72,"sync_messages":"recent"}'
npx supabase functions invoke chatwoot-repair-conversations --body
'{"conversation_ids":[],"repair_messages":true}'
npx supabase functions invoke send-scheduled-reports --body '{"source":"manual-test","dry_run":true}'
```

9.5 SQL de validacion

Validaciones posteriores

```
-- Confirmar tablas principales del schema cw.
select table_name
from information_schema.tables
where table_schema = 'cw'
order by table_name;

-- Confirmar que el dashboard tiene tablas base creadas.
select
  to_regclass('cw.conversations_current') as conversations_current,
  to_regclass('cw.messages') as messages,
  to_regclass('cw.dashboard_tag_settings') as dashboard_tag_settings,
  to_regclass('cw.automated_reports') as automated_reports,
  to_regclass('cw.automated_report_runs') as automated_report_runs;

-- Confirmar extensiones requeridas.
select extname
from pg_extension
where extname in ('pgcrypto', 'pg_net', 'pg_cron', 'supabase_vault')
order by extname;

-- Confirmar cron configurado.
select jobid, jobname, schedule, active
from cron.job
where jobname in ('sync-chatwoot-diario', 'send-scheduled-reports')
order by jobname;

-- Confirmar secretos de Vault sin revelar valores.
select name, created_at
from vault.secrets
where name in ('chatwoot_sync_project_url', 'chatwoot_sync_jwt')
order by name;

-- Confirmar ultima sincronizacion.
select id, sync_type, status, started_at, finished_at, stats, error_message
from cw.sync_runs
order by started_at desc
limit 5;
```

10. Edge Functions

Funcion	Proposito	Variables	Prueba minima
chatwoot-sync	Sincroniza conversaciones, contactos, mensajes, inboxes, atributos, etiquetas y metricas desde Chatwoot. Es la funcion principal para cargar historico y refrescar estado actual.	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, VITE_CHATWOOT_BASE_URL, VITE_CHATWOOT_ACCOUNT_ID, VITE_CHATWOOT_API_TOKEN	Invocarla manualmente con ventana corta y validar cw.sync_runs en success.

Funcion	Proposito	Variables	Prueba minima
chatwoot-label-webhook	Recibe eventos de etiquetas desde Chatwoot para registrar cambios comerciales sin esperar al cierre nocturno.	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, CHATWOOT_WEBHOOK_SECRET	Enviar webhook de prueba desde Chatwoot y confirmar conversation_label_events.
chatwoot-repair-conversations	Relee conversaciones puntuales o ventanas de datos para reparar mensajes, atributos o labels que falten.	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, CHATWOOT_BASE_URL/VITE_CHATWOOT_BASE_URL, CHATWOOT_ACCOUNT_ID/VITE_CHATWOOT_ACCOUNT_ID, CHATWOOT_API_TOKEN/VITE_CHATWOOT_API_TOKEN	Ejecutar sobre IDs concretos y revisar conversaciones/messages actualizados.
send-scheduled-reports	Lee reportes programados activos, genera adjuntos Excel/PDF/CSV y los envia por Resend.	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, RESEND_API_KEY, RESEND_FROM_EMAIL	Crear reporte de prueba, invocar dry_run y revisar automated_report_runs.

11. Chatwoot y Resend

11.1 Chatwoot

- Crear o identificar cuenta Chatwoot y anotar Account ID.
- Crear token API desde perfil/admin con permisos suficientes para conversaciones, contactos, mensajes, inboxes, labels y custom attributes.
- Crear etiquetas comerciales usadas por el embudo: confianza, urgencia, seguimiento, cita, venta, descalificacion u otras segun la empresa.
- Crear atributos personalizados necesarios: score_interes, monto_operacion, fecha_monto_operacion, campana, responsable y campos de datos del lead si aplican.
- Configurar webhook hacia https://<PROJECT_REF>.supabase.co/functions/v1/chatwoot-label-webhook con el secreto definido en CHATWOOT_WEBHOOK_SECRET.
- Probar con una conversacion real y validar que cambien labels, mensajes y estado actual en cw.conversations_current.

11.2 Resend

- Crear cuenta Resend y verificar dominio o remitente autorizado.
- Crear API key exclusiva para reportes.
- Definir RESEND_FROM_EMAIL con nombre visible y correo verificado.
- Probar envio con un reporte programado de baja frecuencia antes de activar reportes para clientes.
- Si Resend rechaza adjuntos o dominio, revisar estado de verificacion y logs de send-scheduled-reports.

12. Reportes y evidencia

Reporte	Formato	Criterio administrativo
Manual por pestana	Excel, PDF, CSV	Debe reflejar lo visible en la pestana y agregar contexto util sin ruido.
Programado	Adjuntos por correo	Debe usar la misma logica conceptual que la descarga manual.
Seguimiento	Excel bloqueado en estructura	No cambiar formato Excel existente; PDF/CSV pueden mejorar.
Conversaciones	Excel bloqueado en estructura	No cambiar formato Excel existente; PDF/CSV pueden mejorar.
Cambios relevantes	Seccion condicional	Solo mostrar si hay cambios con impacto en ventas, montos, citas o metricas visibles.

12.1 Evidencias que debe revisar soporte

- cw.sync_runs para saber si la sincronizacion termino correctamente.
- cw.raw_ingest para revisar payloads crudos si un dato no aparece.

- cw.automated_report_runs para validar envios de reportes.
- Logs de Edge Functions en Supabase.
- cron.job y cw.sync_health para verificar automatizaciones.

13. Riesgos, controles y troubleshooting

Riesgo	Sintoma	Control
Variables de Vercel incompletas	Dashboard carga sin datos o falla API.	Revisar Project Settings y redeploy.
Token Chatwoot invalido	chatwoot-sync termina error.	Regenerar token y actualizar secretos.
Cron caido	No se actualiza historico diario.	Revisar pg_cron, pg_net y cw.sync_health.
Resend sin dominio verificado	Reportes no llegan.	Verificar dominio/remitente y logs.
Etiquetas mal configuradas	KPIs no coinciden con operacion.	Revisar panel admin y dashboard_tag_settings.
Datos recientes incompletos	Hoy/ayer no reflejan Chatwoot.	Ejecutar sync manual o repair.
Migracion faltante	Tabla o columna no existe.	Aplicar SQL base y migraciones en orden.

Anexo A. Catalogo tecnico del schema cw

Objeto	Proposito	Escribe	Lee	Impacto
cw.account_config	Datos generales de la empresa y cuenta Chatwoot.	Configuracion inicial / admin	Funciones y validaciones	Replicacion y soporte
cw.sync_cursor	Guarda el punto hasta donde llego la sincronizacion.	chatwoot-sync	chatwoot-sync	Evita duplicar ventanas de lectura
cw.sync_runs	Bitacora de cada sincronizacion.	chatwoot-sync	Admin / soporte / sync_health	Evidencia de ejecucion
cw.raw_ingest	Payload crudo recibido de API, webhook o reparacion.	Edge Functions	Soporte tecnico	Trazabilidad
cw.inboxes	Canales/inboxes detectados en Chatwoot.	chatwoot-sync	Dashboard y reportes	Filtros por canal
cw.teams	Equipos de Chatwoot.	chatwoot-sync	Operacion / responsables	Carga operativa
cw.attribute_definitions	Definiciones de atributos personalizados.	chatwoot-sync	Admin calidad / reportes	Campos configurables
cw.contacts_current	Estado actual de contactos.	chatwoot-sync / repair	Dashboard / conversaciones	Datos actuales del lead
cw.contact_inboxes	Relacion contacto-inbox.	chatwoot-sync	Canales	Origen del lead
cw.conversations_current	Estado actual de conversaciones y campos comerciales.	chatwoot-sync / webhook / repair	Todas las pestanas	Fuente central de KPIs
cw.contact_attribute_history	Cambios historicos de atributos de contacto.	chatwoot-sync / repair	Cambios relevantes	Explicacion de cambios
cw.conversation_attribute_history	Cambios historicos de atributos de conversacion.	chatwoot-sync / repair	Cambios relevantes	Montos, score, campana
cw.conversation_label_history	Historial simple de etiquetas agregadas o retiradas.	sync / webhook	Embudo y soporte	Evolucion de etapas
cw.business_stage_history	Historial de etapa comercial.	sync / bot / repair	Embudo / rendimiento	Conversion entre estados
cw.messages	Mensajes normalizados por conversacion.	chatwoot-sync / repair	Conversaciones, operacion	Respuesta, ultima interaccion
cw.reporting_events	Eventos de reporteria de Chatwoot.	chatwoot-sync	Operacion	Tiempos y eventos
cw.daily_metrics	Metricas historicas diarias calculadas.	sync nocturno	Tendencias	Historico cerrado
cw.automated_reports	Configuracion de reportes programados.	Panel Reportes	send-scheduled-reports	Correos automaticos
cw.dashboard_tag_settings	Configuracion admin de etiquetas, calidad y columnas.	Panel Admin	Dashboard/reportes	Reglas de negocio
cw.conversation_label_events	Eventos deduplicados de etiquetas.	webhook/sync diff	Cambios relevantes	Senales comerciales
cw.automated_report_runs	Resultado de cada corrida programada.	send-scheduled-reports	Admin/soporte	Evidencia de envio
cw.sync_health	Vista de salud de sync/cron.	SQL view	Soporte	Diagnostico rapido
cw.commercial_audit_events	Vista de cambios comerciales relevantes.	SQL view	Reportes	Explicacion de diferencias

Anexo B. Migraciones del repositorio

Estas migraciones se aplican despues del script base o en el orden natural del repo. Antes de cada bloque se indica el objetivo administrativo. Los valores de proyecto se muestran como placeholders.

20260421134500_create_conversation_label_events.sql

Crea eventos deduplicados de etiquetas para explicar cambios de etapa o senales comerciales.

SQL - 20260421134500_create_conversation_label_events.sql

```
create schema if not exists cw;

create table if not exists cw.conversation_label_events (
  id bigint generated by default as identity primary key,
  chatwoot_conversation_id bigint not null,
  previous_labels text[] not null default '{}':text[],
  next_labels text[] not null default '{}':text[],
  added_labels text[] not null default '{}':text[],
```

```

    removed_labels text[] not null default '{}':text[],
    event_source text not null default 'sync_diff',
    occurred_at timestamp with time zone not null default now(),
    detected_at timestamp with time zone not null default now(),
    raw_payload jsonb not null default '{}':jsonb,
    event_key text unique,
    created_at timestamp with time zone not null default now(),
    constraint conversation_label_events_source_check
        check (event_source = any (array['dashboard', 'webhook', 'sync_diff', 'repair']))
);

create index if not exists conversation_label_events_conversation_idx
on cw.conversation_label_events (chatwoot_conversation_id);

create index if not exists conversation_label_events_occurred_idx
on cw.conversation_label_events (occurred_at);

create index if not exists conversation_label_events_added_gin_idx
on cw.conversation_label_events using gin (added_labels);

create index if not exists conversation_label_events_removed_gin_idx
on cw.conversation_label_events using gin (removed_labels);

```

20260426190000_harden_chatwoot_snapshots_and_sync.sql

Endurece snapshots actuales, agrega vistas de salud, índices, grants y cron diario de sync.

SQL - 20260426190000_harden_chatwoot_snapshots_and_sync.sql

```

-- Chatwoot current-state snapshots and reliable daily sync.
-- Additive only: keeps historical data and recalculates denormalized fields
-- from conversation attrs first, contact attrs second, legacy snapshot last.

alter table cw.conversations_current
add column if not exists conversation_custom_attributes jsonb not null default '{}':jsonb,
add column if not exists contact_custom_attributes jsonb not null default '{}':jsonb;

with prepared as (
select
    chatwoot_conversation_id,
    case
        when jsonb_typeof(contact_custom_attributes) = 'object' and contact_custom_attributes <>
'{}':jsonb then contact_custom_attributes
        when jsonb_typeof(raw_payload #> '{meta,sender,custom_attributes}') = 'object' then
raw_payload #> '{meta,sender,custom_attributes}'
        when jsonb_typeof(meta #> '{sender,custom_attributes}') = 'object' then meta #>
'{}sender,custom_attributes'
        else '{}':jsonb
    end as next_contact_attrs,
    case
        when jsonb_typeof(conversation_custom_attributes) = 'object' and
conversation_custom_attributes <> '{}':jsonb then conversation_custom_attributes
        when jsonb_typeof(raw_payload -> 'custom_attributes') = 'object' then raw_payload ->
'custom_attributes'
        else '{}':jsonb
    end as next_conversation_attrs,
    case
        when jsonb_typeof(custom_attributes) = 'object' then custom_attributes
        else '{}':jsonb
    end as legacy_snapshot
from cw.conversations_current
),
resolved as (
select
    chatwoot_conversation_id,
    next_contact_attrs,
    next_conversation_attrs,
    legacy_snapshot || next_contact_attrs || next_conversation_attrs as resolved_attrs
from prepared
)
update cw.conversations_current c
set
    contact_custom_attributes = r.next_contact_attrs,
    conversation_custom_attributes = r.next_conversation_attrs,
    custom_attributes = r.resolved_attrs,
    nombre_completo = nullif(r.resolved_attrs -> 'nombre_completo', ''),
    fecha_visita = nullif(r.resolved_attrs -> 'fecha_visita', ''),
    hora_visita = nullif(r.resolved_attrs -> 'hora_visita', ''),
    agencia = nullif(r.resolved_attrs -> 'agencia', ''),
    celular = nullif(r.resolved_attrs -> 'celular', ''),
    correo = nullif(r.resolved_attrs -> 'correo', ''),
    campana = nullif(r.resolved_attrs -> 'campana', ''),
    ciudad = nullif(r.resolved_attrs -> 'ciudad', ''),

    edad = nullif(r.resolved_attrs -> 'edad', ''),
    canal = nullif(r.resolved_attrs -> 'canal', ''),
    agente = case
        when lower(coalesce(r.resolved_attrs -> 'agente', '')) in ('true', '1', 'yes', 'si', 'sí') then
true
        when lower(coalesce(r.resolved_attrs -> 'agente', '')) in ('false', '0', 'no') then false
        else c.agente
    end,
    score_interes = case
        when nullif(r.resolved_attrs -> 'score_interes', '') ~ '^-[0-9]+(\.[0-9]+)?$'
        then (r.resolved_attrs -> 'score_interes')::numeric
        else c.score_interes
    end,
end,

```

```

        monto_operacion = coalesce(nullif(r.resolved_attrs ->> 'monto_operacion', ''), c.monto_operacion),
        fecha_monto_operacion = case
            when nullif(r.resolved_attrs ->> 'fecha_monto_operacion', '') ~ '^[0-9]{4}-[0-9]{2}-[0-9]{2}'
            then (r.resolved_attrs ->> 'fecha_monto_operacion')::timestampz
            else c.fecha_monto_operacion
        end,
        updated_at = now()
    from resolved r
    where c.chatwoot_conversation_id = r.chatwoot_conversation_id;

create index if not exists idx_cw_conversations_current_created_at_chatwoot
    on cw.conversations_current (created_at_chatwoot desc);

create index if not exists idx_cw_conversations_current_last_activity_at_chatwoot
    on cw.conversations_current (last_activity_at_chatwoot desc);

create index if not exists idx_cw_conversations_current_updated_at_chatwoot
    on cw.conversations_current (updated_at_chatwoot desc);

create index if not exists idx_cw_conversations_current_fecha_monto_operacion
    on cw.conversations_current (fecha_monto_operacion desc);

create index if not exists idx_cw_conversations_current_labels_gin
    on cw.conversations_current using gin (labels);

create index if not exists idx_cw_conversations_current_custom_attrs_gin
    on cw.conversations_current using gin (custom_attributes);

create index if not exists idx_cw_conversations_current_conversation_attrs_gin
    on cw.conversations_current using gin (conversation_custom_attributes);

create index if not exists idx_cw_conversations_current_contact_attrs_gin
    on cw.conversations_current using gin (contact_custom_attributes);

create index if not exists idx_cw_conversations_current_resolved_monto
    on cw.conversations_current ((custom_attributes ->> 'monto_operacion'));

create index if not exists idx_cw_messages_created_at_chatwoot
    on cw.messages (created_at_chatwoot desc);

create or replace view cw.sync_health as

with latest_run as (
    select *
    from cw.sync_runs
    order by started_at desc
    limit 1
),
latest_net as (
    select
        id,
        status_code,
        content,
        timed_out,
        error_msg,
        created
    from net._http_response
    order by created desc
    limit 1
),
active_job as (
    select
        jobid,
        schedule,
        active
    from cron.job
    where jobname = 'sync-chatwoot-diario'
    order by jobid desc
    limit 1
)
select
    lr.id as latest_sync_run_id,
    lr.sync_type,
    lr.status as latest_sync_status,
    lr.started_at,
    lr.finished_at,
    extract(epoch from (coalesce(lr.finished_at, now()) - lr.started_at))::integer as duration_seconds,
    lr.stats,
    lr.error_message,
    nullif(lr.stats ->> 'pages', '')::integer as pages_count,
    nullif(lr.stats ->> 'conversations', '')::integer as conversations_count,
    nullif(lr.stats ->> 'contacts', '')::integer as contacts_count,
    nullif(lr.stats ->> 'messages', '')::integer as messages_count,
    ln.id as latest_pg_net_request_id,
    ln.status_code as latest_pg_net_status_code,
    left(ln.content, 700) as latest_pg_net_content,
    ln.timed_out as latest_pg_net_timed_out,
    ln.error_msg as latest_pg_net_error,
    ln.created as latest_pg_net_created_at,
    aj.jobid as cron_job_id,
    aj.schedule as cron_schedule,
    aj.active as cron_active
from (select 1) anchor
left join latest_run lr on true
left join latest_net ln on true
left join active_job aj on true;

do $$
declare
    table_name text;

```

```

dashboard_tables text[] := array[
    'account_config',
    'sync_cursor',
    'sync_runs',
    'inboxes',
    'teams',
    'attribute_definitions',
    'contacts_current',
    'contact_inboxes',
    'conversations_current',
    'messages',
    'reporting_events',
    'daily_metrics',
    'automated_reports',
    'dashboard_tag_settings',
    'conversation_label_events'
];
protected_tables text[] := array[
    'raw_ingest',
    'contact_attribute_history',
    'conversation_attribute_history',
    'conversation_label_history',
    'business_stage_history'
];
begin
    foreach table_name in array dashboard_tables loop
        execute format('alter table cw.%I enable row level security', table_name);

        if not exists (
            select 1
            from pg_policies
            where schemaname = 'cw'
            and tablename = table_name
            and policyname = 'authenticated_all'
        ) then
            execute format(
                'create policy authenticated_all on cw.%I for all to authenticated using (true) with
check (true)',
                table_name
            );
        end if;
    end loop;

    foreach table_name in array protected_tables loop
        execute format('alter table cw.%I enable row level security', table_name);
    end loop;
end $$;

revoke all on all tables in schema cw from anon;
grant usage on schema cw to authenticated;

grant select, insert, update, delete on all tables in schema cw to authenticated;
grant usage, select on all sequences in schema cw to authenticated;
grant select on cw.sync_health to authenticated;

do $$
declare
    existing_command text;
    existing_bearer text;
begin
    select command
    into existing_command
    from cron.job
    where jobname = 'sync-chatwoot-diario'
    order by jobid desc
    limit 1;

    existing_bearer := substring(coalesce(existing_command, '') from
'Bearer[[:space:]]+([A-Za-z0-9_-.]+)');

    if not exists (select 1 from vault.secrets where name = 'chatwoot_sync_project_url') then
        perform vault.create_secret(
            'https://<PROJECT_REF>.supabase.co',
            'chatwoot_sync_project_url',
            'Supabase project URL used by pg_cron to invoke chatwoot-sync'
        );
    end if;

    if existing_bearer is not null
    and not exists (select 1 from vault.secrets where name = 'chatwoot_sync_jwt') then
        perform vault.create_secret(
            existing_bearer,
            'chatwoot_sync_jwt',
            'JWT used by pg_cron to invoke chatwoot-sync'
        );
    end if;
end $$;

do $$
declare
    job record;
begin
    for job in
        select jobid
        from cron.job
        where jobname = 'sync-chatwoot-diario'
    loop
        perform cron.unschedule(job.jobid);
    end loop;
end $$;

```

```

select cron.schedule(
  'sync-chatwoot-diario',
  '1 5 * * *',
  $$
    select
      net.http_post(
        url := (select decrypted_secret from vault.decrypted_secrets where name =
'chatwoot_sync_project_url') || '/functions/v1/chatwoot-sync',
        headers := jsonb_build_object(
          'Content-Type', 'application/json',
          'Authorization', 'Bearer ' || (select decrypted_secret from vault.decrypted_secrets where
name = 'chatwoot_sync_jwt')
        ),
        body := '{"mode":"full","window_hours":72,"sync_messages":"recent"}'::jsonb,
        timeout_milliseconds := 60000
      ) as request_id;
    $$
)
where not exists (
  select 1
  from cron.job
  where jobname = 'sync-chatwoot-diario'
);

```

20260426202500_grant_cw_to_service_role.sql

Garantiza permisos del service_role sobre schema cw para Edge Functions.

SQL - 20260426202500_grant_cw_to_service_role.sql

```

-- Edge Functions use the Supabase service role key. RLS bypass still requires
-- table privileges on non-public schemas, so grant cw access to service_role
-- without reopening anon access.

grant usage on schema cw to service_role;
grant select, insert, update, delete on all tables in schema cw to service_role;
grant usage, select on all sequences in schema cw to service_role;

```

20260427165000_reporting_exports_and_scheduled_runs.sql

Amplia reportes programados, corridas y cron de envio cada 5 minutos.

SQL - 20260427165000_reporting_exports_and_scheduled_runs.sql

```

alter table cw.automated_reports
  add column if not exists report_scope text not null default 'tab',
  add column if not exists tab_ids text[] not null default '{}'::text[],
  add column if not exists critical_profile_key text,
  add column if not exists file_formats text[] not null default array['excel']::text[],
  add column if not exists date_range_mode text not null default 'closed_period',
  add column if not exists filters jsonb not null default '{}'::jsonb,
  add column if not exists created_by uuid references auth.users(id) on delete set null,
  add column if not exists created_by_email text,
  add column if not exists last_status text,
  add column if not exists last_error text;

do $$
begin
  if not exists (
    select 1
    from pg_constraint
    where conname = 'automated_reports_scope_check'
    and conrelid = 'cw.automated_reports'::regclass
  ) then
    alter table cw.automated_reports
      add constraint automated_reports_scope_check
      check (report_scope in ('tab', 'critical_profile')) not valid;
  end if;
end $$;

alter table cw.automated_reports
  validate constraint automated_reports_scope_check;

do $$
begin
  if not exists (
    select 1
    from pg_constraint
    where conname = 'automated_reports_date_range_mode_check'
    and conrelid = 'cw.automated_reports'::regclass
  ) then
    alter table cw.automated_reports
      add constraint automated_reports_date_range_mode_check
      check (date_range_mode in ('closed_period', 'current_filters')) not valid;
  end if;
end $$;

alter table cw.automated_reports
  validate constraint automated_reports_date_range_mode_check;

create index if not exists automated_reports_active_schedule_idx
  on cw.automated_reports (is_active, frequency, schedule_time);

create index if not exists automated_reports_scope_idx

```

```

on cw.automated_reports (report_scope, critical_profile_key);

create table if not exists cw.automated_report_runs (
  id uuid primary key default gen_random_uuid(),
  automated_report_id uuid references cw.automated_reports(id) on delete set null,
  status text not null default 'running' check (status in ('running', 'success', 'error', 'skipped')),
  recipients text,
  file_formats text[] not null default '{}':text[],
  report_scope text,
  tab_ids text[] not null default '{}':text[],
  critical_profile_key text,
  scheduled_for timestampz,
  started_at timestampz not null default now(),
  finished_at timestampz,
  error_message text,
  metadata jsonb not null default '{}':jsonb
);

create index if not exists automated_report_runs_report_started_idx
on cw.automated_report_runs (automated_report_id, started_at desc);

create index if not exists automated_report_runs_status_started_idx
on cw.automated_report_runs (status, started_at desc);

alter table cw.automated_reports enable row level security;
alter table cw.automated_report_runs enable row level security;

drop policy if exists "authenticated manage automated reports" on cw.automated_reports;
create policy "authenticated manage automated reports"
on cw.automated_reports
for all
to authenticated
using (true)
with check (true);

drop policy if exists "authenticated read automated report runs" on cw.automated_report_runs;
create policy "authenticated read automated report runs"
on cw.automated_report_runs
for select
to authenticated
using (true);

grant usage on schema cw to authenticated, service_role;
grant select, insert, update, delete on cw.automated_reports to authenticated, service_role;
grant select on cw.automated_report_runs to authenticated;
grant all on cw.automated_report_runs to service_role;

do $$
declare
  job record;
begin
  for job in
    select jobid
    from cron.job
    where jobname = 'send-scheduled-reports'
  loop
    perform cron.unschedule(job.jobid);
  end loop;
end $$;

select cron.schedule(
  'send-scheduled-reports',
  '*/*5 * * * *',
  $$
select
  net.http_post(
    url := (select decrypted_secret from vault.decrypted_secrets where name =
'chatwoot_sync_project_url') || '/functions/v1/send-scheduled-reports',
    headers := jsonb_build_object(
      'Content-Type', 'application/json',
      'Authorization', 'Bearer ' || (select decrypted_secret from vault.decrypted_secrets where
name = 'chatwoot_sync_jwt')
    ),
    body := jsonb_build_object('source', 'pg_cron'),
    timeout_milliseconds := 60000
  ) as request_id;
  $$
);

```

20260427220500_backfill_chatwoot_inbox_channel.sql

Rellena canal/inbox desde payloads historicos de Chatwoot.

SQL - 20260427220500_backfill_chatwoot_inbox_channel.sql

```

-- Backfill denormalized channel labels from the authoritative Chatwoot inbox.
-- This keeps contact/conversation custom attribute snapshots intact and only fixes
-- the resolved snapshot used by dashboard KPIs, tables and automated reports.

with channel_sources as (
  select
    c.chatwoot_conversation_id,
    c.custom_attributes,
    lower(coalesce(c.raw_payload ->> 'channel_type', '')) as raw_channel_type,
    lower(coalesce(i.channel_type, '')) as inbox_channel_type,
    lower(concat_ws(

```

```

        ' ',
        c.raw_payload ->> 'channel_type',
        c.raw_payload ->> 'channel_name',
        c.raw_payload ->> 'source',
        c.raw_payload ->> 'provider',
        c.raw_payload #>> '{additional_attributes,channel}',
        c.raw_payload #>> '{additional_attributes,social_channel}',
        c.raw_payload #>> '{meta,sender,additional_attributes,channel}',
        c.raw_payload #>> '{meta,sender,additional_attributes,social_channel}',
        c.raw_payload #>> '{meta,sender,additional_attributes,provider}',
        c.raw_payload #>> '{meta,sender,additional_attributes,platform}',
        c.raw_payload #>> '{inbox,name}',
        c.raw_payload #>> '{inbox,channel_type}',
        c.raw_payload #>> '{inbox,website_url}',
        i.name,
        i.channel_type,
        i.website_url,
        c.conversation_custom_attributes ->> 'canal',
        c.contact_custom_attributes ->> 'canal',
        c.custom_attributes ->> 'canal',
        c.canal
    )) as fallback_hints
from cw.conversations_current c
left join cw.inboxes i
    on i.chatwoot_inbox_id = c.chatwoot_inbox_id
),
resolved as (
    select
        chatwoot_conversation_id,
        custom_attributes,
        case
            when raw_channel_type like '%whatsapp%' or inbox_channel_type like '%whatsapp%' then
                'WhatsApp'
            when raw_channel_type like '%instagram%' or inbox_channel_type like '%instagram%' then
                'Instagram'
            when raw_channel_type like '%facebook%' or raw_channel_type like '%messenger%'
                or inbox_channel_type like '%facebook%' or inbox_channel_type like '%messenger%' then
                'Facebook'
            when raw_channel_type like '%telegram%' or inbox_channel_type like '%telegram%' then
                'Telegram'
            when raw_channel_type like '%tiktok%' or inbox_channel_type like '%tiktok%' then 'TikTok'
            when raw_channel_type like '%webwidget%' or raw_channel_type like '%web_widget%' or
                raw_channel_type like '%website%'
                or inbox_channel_type like '%webwidget%' or inbox_channel_type like '%web_widget%' or
                inbox_channel_type like '%website%' then 'Sitio web'
            when fallback_hints like '%whatsapp%' or fallback_hints like '%wa.me%' then 'WhatsApp'
            when fallback_hints like '%instagram%' then 'Instagram'
            when fallback_hints like '%facebook%' or fallback_hints like '%messenger%' then 'Facebook'
            when fallback_hints like '%telegram%' or fallback_hints like '%t.me%' then 'Telegram'
            when fallback_hints like '%tiktok%' or fallback_hints like '%tik tok%' then 'TikTok'
            when fallback_hints like '%webwidget%' or fallback_hints like '%web_widget%'
                or fallback_hints like '%web widget%' or fallback_hints like '%website%'
                or fallback_hints like '%web site%' or fallback_hints like '%sitio web%'
                or fallback_hints like '%pagina web%' or fallback_hints like '%livechat%'
                or fallback_hints like '%live chat%' then 'Sitio web'
            else null
        end as resolved_channel
    from channel_sources
)
update cw.conversations_current c
set
    canal = r.resolved_channel,
    custom_attributes = jsonb_set(
        case
            when jsonb_typeof(coalesce(c.custom_attributes, '{} '::jsonb)) = 'object'
            then coalesce(c.custom_attributes, '{} '::jsonb)
            else '{} '::jsonb
        end,
        '{canal}',
        to_jsonb(r.resolved_channel),
        true
    ),
    updated_at = now()
from resolved r
where c.chatwoot_conversation_id = r.chatwoot_conversation_id
and r.resolved_channel is not null
and (
    c.canal is distinct from r.resolved_channel
    or lower(coalesce(c.custom_attributes ->> 'canal', '')) in ('', 'otro', 'other', 'unknown', 'sin
canal', 'n/a', 'na')
);

```

20260427225200_enforce_chatwoot_inbox_channel.sql

Refuerza la normalizacion de canal/inbox para que filtros y reportes coincidan.

SQL - 20260427225200_enforce_chatwoot_inbox_channel.sql

```

-- Keep the resolved dashboard channel coherent even if an older writer omits it.
-- Chatwoot inbox metadata is the authoritative source for the transport channel.

create or replace function cw.resolve_chatwoot_channel_label(
    p_raw_payload jsonb,
    p_custom_attributes jsonb,
    p_contact_custom_attributes jsonb,
    p_conversation_custom_attributes jsonb,

```

```

        p_current_canal text,
        p_inbox_id bigint
    )
    returns text
    language plpgsql
    stable
    as $$
    declare
        v_inbox record;
        v_raw_channel_type text := lower(coalesce(p_raw_payload ->> 'channel_type', ''));
        v_inbox_channel_type text := '';
        v_hints text;
        v_clean_current text := nullif(btrim(coalesce(p_current_canal, '')), '');
        v_clean_attr text := nullif(btrim(coalesce(
            p_conversation_custom_attributes ->> 'canal',
            p_contact_custom_attributes ->> 'canal',
            p_custom_attributes ->> 'canal',
            ''
        )), '');
    begin
        select i.name, i.channel_type, i.website_url, i.website_token
        into v_inbox
        from cw.inboxes i
        where i.chatwoot_inbox_id = p_inbox_id
        limit 1;

        v_inbox_channel_type := lower(coalesce(v_inbox.channel_type, ''));

        v_hints := lower(concat_ws(
            '',
            p_raw_payload ->> 'channel_type',
            p_raw_payload ->> 'channel_name',
            p_raw_payload ->> 'source',
            p_raw_payload ->> 'provider',
            p_raw_payload #>> '{additional_attributes,channel}',
            p_raw_payload #>> '{additional_attributes,social_channel}',
            p_raw_payload #>> '{meta,sender,additional_attributes,channel}',
            p_raw_payload #>> '{meta,sender,additional_attributes,social_channel}',
            p_raw_payload #>> '{meta,sender,additional_attributes,provider}',
            p_raw_payload #>> '{meta,sender,additional_attributes,platform}',
            p_raw_payload #>> '{inbox,name}',
            p_raw_payload #>> '{inbox,channel_type}',
            p_raw_payload #>> '{inbox,website_url}',
            v_inbox.name,
            v_inbox.channel_type,
            v_inbox.website_url,

            v_inbox.website_token,
            v_clean_attr,
            v_clean_current
        ));

        return case
            when v_raw_channel_type like '%whatsapp%' or v_inbox_channel_type like '%whatsapp%' then
                'WhatsApp'
            when v_raw_channel_type like '%instagram%' or v_inbox_channel_type like '%instagram%' then
                'Instagram'
            when v_raw_channel_type like '%facebook%' or v_inbox_channel_type like '%messenger%'
                or v_inbox_channel_type like '%facebook%' or v_inbox_channel_type like '%messenger%' then
                'Facebook'
            when v_raw_channel_type like '%telegram%' or v_inbox_channel_type like '%telegram%' then
                'Telegram'
            when v_raw_channel_type like '%tiktok%' or v_inbox_channel_type like '%tiktok%' then 'TikTok'
            when v_raw_channel_type like '%webwidget%' or v_raw_channel_type like '%web_widget%' or
                v_raw_channel_type like '%website%'
                or v_inbox_channel_type like '%webwidget%' or v_inbox_channel_type like '%web_widget%' or
                v_inbox_channel_type like '%website%' then 'Sitio web'
            when v_hints like '%whatsapp%' or v_hints like '%wa.me%' then 'WhatsApp'
            when v_hints like '%instagram%' then 'Instagram'
            when v_hints like '%facebook%' or v_hints like '%messenger%' then 'Facebook'
            when v_hints like '%telegram%' or v_hints like '%t.me%' then 'Telegram'
            when v_hints like '%tiktok%' or v_hints like '%tik tok%' then 'TikTok'
            when v_hints like '%webwidget%' or v_hints like '%web_widget%'
                or v_hints like '%web widget%' or v_hints like '%website%'
                or v_hints like '%web site%' or v_hints like '%sitio web%'
                or v_hints like '%pagina web%' or v_hints like '%livechat%'
                or v_hints like '%live chat%' then 'Sitio web'
            when lower(coalesce(v_clean_current, '')) not in ('', 'otro', 'other', 'unknown', 'sin canal',
                'n/a', 'na') then v_clean_current
            when lower(coalesce(v_clean_attr, '')) not in ('', 'otro', 'other', 'unknown', 'sin canal',
                'n/a', 'na') then v_clean_attr
            else null
        end;
    end;
    $$;

    create or replace function cw.set_resolved_chatwoot_channel()
    returns trigger
    language plpgsql
    as $$
    declare
        v_resolved_channel text;
    begin
        v_resolved_channel := cw.resolve_chatwoot_channel_label(
            coalesce(new.raw_payload, '{}')::jsonb,
            coalesce(new.custom_attributes, '{}')::jsonb,
            coalesce(new.contact_custom_attributes, '{}')::jsonb,
            coalesce(new.conversation_custom_attributes, '{}')::jsonb,
            new.canal,
            new.chatwoot_inbox_id
        );
    end;

```

```

);
if v_resolved_channel is not null then
    new.canal := v_resolved_channel;
    new.custom_attributes := jsonb_set(
        case
            when jsonb_typeof(coalesce(new.custom_attributes, '{}')::jsonb) = 'object'
            then coalesce(new.custom_attributes, '{}')::jsonb
            else '{}'::jsonb
        end,
        '{canal}',
        to_jsonb(v_resolved_channel),
        true
    );
end if;

return new;
end;
$$;

drop trigger if exists conversations_current_resolve_channel_biu on cw.conversations_current;

create trigger conversations_current_resolve_channel_biu
before insert or update of
    chatwoot_inbox_id,
    raw_payload,
    canal,
    custom_attributes,
    contact_custom_attributes,
    conversation_custom_attributes
on cw.conversations_current
for each row
execute function cw.set_resolved_chatwoot_channel();

```

20260429123000_commercial_current_state_audit.sql

Separa estado comercial actual de cambios relevantes historicos para reportes claros.

SQL - 20260429123000_commercial_current_state_audit.sql

```

-- Separate current commercial accounting from historical commercial audit.

alter table cw.conversation_attribute_history
add column if not exists event_key text;

alter table cw.contact_attribute_history
add column if not exists event_key text;

update cw.conversation_attribute_history
set event_key = concat_ws(':', 'conversation_attr', id::text)
where event_key is null;

update cw.contact_attribute_history
set event_key = concat_ws(':', 'contact_attr', id::text)
where event_key is null;

create unique index if not exists conversation_attribute_history_event_key_idx
on cw.conversation_attribute_history (event_key)
where event_key is not null;

create unique index if not exists contact_attribute_history_event_key_idx
on cw.contact_attribute_history (event_key)
where event_key is not null;

with resolved as (
    select
        chatwoot_conversation_id,
        coalesce(contact_custom_attributes, '{}')::jsonb
        || coalesce(conversation_custom_attributes, '{}')::jsonb as attrs
    from cw.conversations_current
)
update cw.conversations_current c
set
    custom_attributes = r.attrs,
    nombre_completo = nullif(r.attrs ->> 'nombre_completo', ''),
    fecha_visita = nullif(r.attrs ->> 'fecha_visita', ''),
    hora_visita = nullif(r.attrs ->> 'hora_visita', ''),
    agencia = nullif(r.attrs ->> 'agencia', ''),
    celular = nullif(r.attrs ->> 'celular', ''),
    correo = nullif(r.attrs ->> 'correo', ''),
    campana = nullif(r.attrs ->> 'campana', ''),
    ciudad = nullif(r.attrs ->> 'ciudad', ''),
    edad = nullif(r.attrs ->> 'edad', ''),
    canal = coalesce(nullif(r.attrs ->> 'canal', ''), c.canal),
    business_stage_current = nullif(r.attrs ->> 'business_stage', ''),
    agente = case
        when lower(coalesce(r.attrs ->> 'agente', '')) in ('true', '1', 'yes', 'si', 'si') then true
        when lower(coalesce(r.attrs ->> 'agente', '')) in ('false', '0', 'no') then false
        else null
    end,
    score_interes = case
        when nullif(r.attrs ->> 'score_interes', '') ~ '^-[0-9]+(\.[0-9]+)?$'
        then (r.attrs ->> 'score_interes')::numeric
        else null
    end,
    monto_operacion = nullif(r.attrs ->> 'monto_operacion', ''),
    fecha_monto_operacion = case

```

```

        when nullif(r.attrs ->> 'fecha_monto_operacion', '') ~ '^[0-9]{4}-[0-9]{2}-[0-9]{2}'
        then (r.attrs ->> 'fecha_monto_operacion')::timestampz
        else null
    end,
    updated_at = now()
from resolved r
where c.chatwoot_conversation_id = r.chatwoot_conversation_id;

create or replace view cw.commercial_audit_events as
with tracked_attribute_events as (
    select
        concat('conversation_attr:', h.id::text) as event_key,
        h.id,
        h.chatwoot_conversation_id,
        case
            when h.attribute_key = 'monto_operacion'
                and h.old_value is not null
                and (h.new_value is null or h.new_value::text in ('null', ''))
            then 'monto_eliminado'
            when h.attribute_key = 'monto_operacion'
            then 'monto_actualizado'
            else 'atributo_comercial_actualizado'
        end as event_type,
        h.attribute_key as field_name,
        h.attribute_key,
        h.old_value as previous_value,
        h.new_value as current_value,
        h.old_value as historical_value,
        h.changed_at,
        h.created_at as detected_at,
        h.change_source,
        case
            when h.attribute_key = 'monto_operacion'
                and h.old_value is not null
                and (h.new_value is null or h.new_value::text in ('null', ''))
            then 'monto_eliminado'
            when h.attribute_key = 'monto_operacion'
            then 'monto_actualizado'
            else 'dato_comercial_actualizado'
        end as business_impact,
        jsonb_build_object('source_table', 'conversation_attribute_history') as raw_payload
    from cw.conversation_attribute_history h
    where h.attribute_key in (
        'monto_operacion',
        'fecha_monto_operacion',
        'score_interes',
        'score',
        'lead_score',
        'puntaje',
        'responsable',
        'campana',

        'utm_campaign',
        'business_stage'
    )
),
tracked_contact_attribute_events as (
    select
        concat('contact_attr:', h.id::text, ':', c.chatwoot_conversation_id::text) as event_key,
        h.id,
        c.chatwoot_conversation_id,
        'atributo_comercial_contacto_actualizado' as event_type,
        h.attribute_key as field_name,
        h.attribute_key,
        h.old_value as previous_value,
        h.new_value as current_value,
        h.old_value as historical_value,
        h.changed_at,
        h.created_at as detected_at,
        h.change_source,
        'dato_comercial_actualizado' as business_impact,
        jsonb_build_object('source_table', 'contact_attribute_history') as raw_payload
    from cw.contact_attribute_history h
    join cw.conversations_current c
    on c.chatwoot_contact_id = h.chatwoot_contact_id
    where h.attribute_key in (
        'monto_operacion',
        'fecha_monto_operacion',
        'score_interes',
        'score',
        'lead_score',
        'puntaje',
        'responsable',
        'campana',
        'utm_campaign'
    )
),
tracked_label_events as (
    select
        concat('label_event:', e.id::text) as event_key,
        e.id,
        e.chatwoot_conversation_id,
        case
            when not (coalesce(c.labels, '{}'::text[]) && array['venta_exitosa', 'venta']::text[])
            and (
                coalesce(e.previous_labels, '{}'::text[]) && array['venta_exitosa', 'venta']::text[]
                or coalesce(e.next_labels, '{}'::text[]) && array['venta_exitosa', 'venta']::text[]
                or coalesce(e.added_labels, '{}'::text[]) && array['venta_exitosa', 'venta']::text[]
                or coalesce(e.removed_labels, '{}'::text[]) && array['venta_exitosa',
'venta']::text[]

```

```

        )
        then 'venta_historica_no_vigente'
        else 'evento_etiqueta_venta'
    end as event_type,
    'labels'::text as field_name,
    'labels'::text as attribute_key,
    to_jsonb(e.previous_labels) as previous_value,
    to_jsonb(e.next_labels) as current_value,
    to_jsonb(e.previous_labels) as historical_value,
    e.occurred_at as changed_at,
    e.detected_at,
    e.event_source as change_source,
    case
        when not (coalesce(c.labels, '{}')::text[] && array['venta_exitosa', 'venta']::text[])
        and (
            coalesce(e.previous_labels, '{}')::text[] && array['venta_exitosa', 'venta']::text[]
            or coalesce(e.next_labels, '{}')::text[] && array['venta_exitosa', 'venta']::text[]
            or coalesce(e.added_labels, '{}')::text[] && array['venta_exitosa', 'venta']::text[]
            or coalesce(e.removed_labels, '{}')::text[] && array['venta_exitosa',
'venta']::text[]
        )
        then 'venta_historica_no_vigente'
        else 'evento_etiqueta_venta'
    end as business_impact,
    coalesce(e.raw_payload, '{}')::jsonb
    || jsonb_build_object('source_table', 'conversation_label_events') as raw_payload
from cw.conversation_label_events e
left join cw.conversations_current c
on c.chatwoot_conversation_id = e.chatwoot_conversation_id
where
    coalesce(e.previous_labels, '{}')::text[] && array['venta_exitosa', 'venta']::text[]
    or coalesce(e.next_labels, '{}')::text[] && array['venta_exitosa', 'venta']::text[]
    or coalesce(e.added_labels, '{}')::text[] && array['venta_exitosa', 'venta']::text[]
    or coalesce(e.removed_labels, '{}')::text[] && array['venta_exitosa', 'venta']::text[]
)
select * from tracked_attribute_events
union all
select * from tracked_contact_attribute_events
union all
select * from tracked_label_events;

grant select on cw.commercial_audit_events to authenticated;
grant select on cw.commercial_audit_events to service_role;

```

Anexo C. Migraciones remotas detectadas

El entorno Supabase actual registra migraciones iniciales que pudieron haberse aplicado directamente antes de consolidar los archivos locales. El script base de este manual representa la estructura necesaria para un entorno nuevo.

Version	Nombre	Accion recomendada
20260411181400	chatwoot_historical_sync_init	Cubierta por SQL base. Exportar desde Supabase si se requiere auditoria exacta.
20260421084504	create_cw_dashboard_tag_settings	Cubierta por SQL base y catalogo.
20260429115733	commercial_audit_events_infer_non_current_sale	Cubierta por vista de cambios relevantes y migracion comercial.
20260429120540	commercial_audit_amount_update_label	Cubierta por vista de cambios relevantes y reportes.

Anexo D. Checklist final de puesta en marcha

Item	OK / No OK / N.A.	Evidencia
Proyecto Supabase creado y SQL aplicado.		
Extensiones pgcrypto, pg_net, pg_cron y Vault habilitadas.		
Edge Functions desplegadas.		
Secretos de Supabase configurados.		
Chatwoot Account ID y API token probados.		
Webhook Chatwoot configurado y probado.		
Vercel conectado a GitHub main con variables VITE_.		
Resend verificado con correo FROM valido.		
Sync inicial ejecutado con success.		
Panel admin configurado para la empresa.		

Item	OK / No OK / N.A.	Evidencia
Dashboard muestra datos en Estrategia, Operacion, Calidad y Conversaciones.		
Reporte manual y programado probados.		